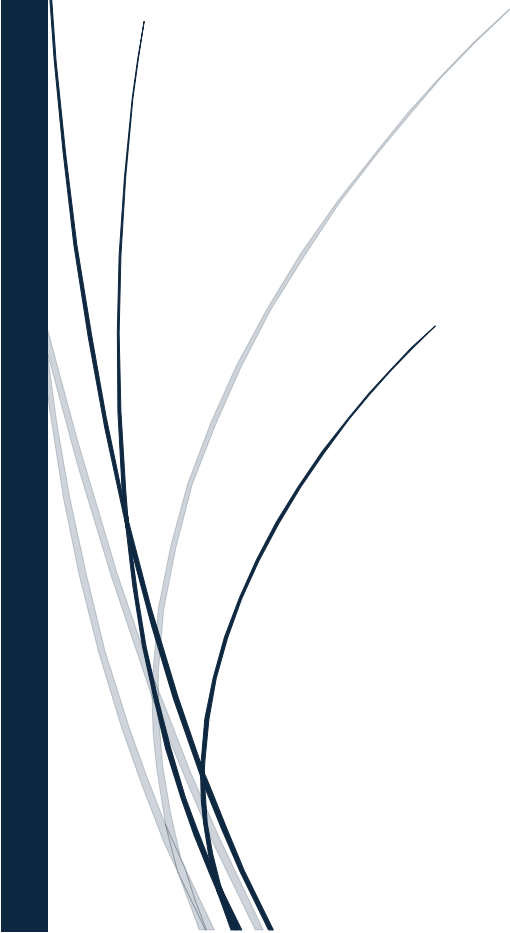




Proyecto 2 – CRM/IoT

Integración de aplicaciones



Marina Amaya, Ignacio Fernández y
Britney Cuichan

Índice:

1. Contexto y objetivo	2
2. Implementación del CRM	2
Endpoints implementados	2
Validación y errores	3
Datos semilla	3
Consideraciones	3
3. Implementación del IoT	3
Endpoints implementados	3
Validación y errores	4
Datos semilla	4
Consideraciones	4
4. Integración y pruebas	5
5. Errores y soluciones	5
6. Conclusión	6

1. Contexto y objetivo

El objetivo del Tema 2 ha sido llevar todo lo que definimos teóricamente en el Tema 1 a una implementación real mediante dos microservicios independientes: un CRM simulado y un IoT simulado.

- En el CRM hemos gestionado la parte administrativa de clientes y pedidos.
- En el IoT hemos manejado sensores y lecturas operativas.

Nuestro propósito ha sido exponer datos válidos a través de APIs REST y validarlos usando los JSON Schema definidos previamente. De esta forma nos aseguramos de que toda la información (clientes, pedidos, sensores y lecturas) sea coherente, correcta y predecible.

2. Implementación del CRM

Tecnología: Node.js (v18+) con Express y AJV para validación de JSON Schema.

Endpoints implementados

```
{
  "total": 50,
  "page": 1,
  "pageSize": 25,
  "data": [
    {
      "clienteId": "C001",
      "nombreCompleto": "Ana García López",
      "email": "ana.garcia@example.com",
      "telefono": "+34 612345678",
      "tipoForfait": "Temporada",
      "saldoMonedero": 120.5,
      "ubicacionId": "U001"
    },
    {
      "clienteId": "C002",
      "nombreCompleto": "Javier Pérez Ramos",
      "email": "javier.perez@example.com",
      "telefono": "+34 623456789",
      "tipoForfait": "Diario",
      "saldoMonedero": 45,
      "ubicacionId": "U002"
    },
    {
      "clienteId": "C003",
      "nombreCompleto": "María Fernández Ruiz",
      "email": "maria.fernandez@example.com",
      "telefono": "+34 634567890",
      "tipoForfait": "Temporada",
      "saldoMonedero": 210.75,
      "ubicacionId": "U001"
    }
  ],
}
```

- GET /clientes: devuelve listado de clientes con filtros opcionales:
 - ?q=: búsqueda por nombre o email (no sensible a mayúsculas).
 - ?ubicacionId=: filtra por ubicación.
 - ?page= y ?pageSize=: paginación.
- GET /clientes/:id: devuelve cliente por id.

```
{
  "total": 10,
  "page": 1,
  "pageSize": 25,
  "data": [
    {
      "pedidoId": "P001",
      "clienteId": "C001",
      "fechaPedido": "2025-02-01T10:00:00Z",
      "totalMonto": 120.5,
      "estado": "Completado"
    },
    {
      "pedidoId": "P002",
      "clienteId": "C001",
      "fechaPedido": "2025-03-10T09:45:00Z",
      "totalMonto": 85,
      "estado": "Pendiente de Pago"
    },
    {
      "pedidoId": "P003",
      "clienteId": "C002",
      "fechaPedido": "2025-04-12T12:30:00Z",
      "totalMonto": 45,
      "estado": "Completado"
    },
    {
      "pedidoId": "P004",
      "clienteId": "C003",
      "fechaPedido": "2025-01-25T08:15:00Z",
      "totalMonto": 210.75,
      "estado": "Cancelado"
    }
  ],
}
```

- GET /pedidos: listado de pedidos con filtros opcionales clienteId y estado.
- GET /pedidos/:id: devuelve pedido por id.

Validación y errores

- Cada respuesta valida contra cliente.schema.json o pedido.schema.json.
- Códigos de estado implementados:
 - 200 éxito.
 - 400 parámetros inválidos.
 - 404 recurso no encontrado.
 - 500 datos no válidos según schema.

Datos semilla

- clientes.json: 50 clientes con distintos tipoForfait y saldoMonedero.
- pedidos.json: 10 pedidos para distintos clientes con estados variados (Completado, Pendiente de Pago, Cancelado).

Consideraciones

- Normalización de texto para búsqueda.
- Paginación segura con límites razonables.
- Respuestas deterministas desde ficheros JSON estáticos.

3. Implementación del IoT

Tecnología: Python 3.11 con FastAPI y jsonschema para validación de lecturas.

Endpoints implementados

```
[
  {
    "id": "REM-TS01",
    "tipo": "aforo",
    "ubicacionId": "ZonaPrincipiantes",
    "estadoOperacional": "Operativo"
  },
  {
    "id": "REM-TC02",
    "tipo": "vibracion",
    "ubicacionId": "ZonaExpertos",
    "estadoOperacional": "Operativo"
  },
  {
    "id": "REM-TS01-TEMP",
    "tipo": "temperatura",
    "ubicacionId": "ZonaPrincipiantes",
    "estadoOperacional": "Operativo"
  }
]
```

- GET /sensores: devuelve listado de sensores, cada uno con:
 - id, tipo (aforo, temperatura, vibración), ubicacionId, estadoOperacional.

```
[
  {
    "id": "LEC-AF-001",
    "sensorId": "REM-TS01",
    "valor": 150,
    "unidad": "personas",
    "timestamp": "2025-10-16T10:00:00Z"
  },
  {
    "id": "LEC-VIB-001",
    "sensorId": "REM-TC02",
    "valor": 0.5,
    "unidad": "mm/s",
    "timestamp": "2025-10-16T10:01:00Z"
  },
  {
    "id": "LEC-TEMP-001",
    "sensorId": "REM-TS01-TEMP",
    "valor": -2.5,
    "unidad": "Â°C",
    "timestamp": "2025-10-16T10:02:00Z"
  },
  {
    "id": "LEC-AF-002",
    "sensorId": "REM-TS01",
    "valor": 155,
    "unidad": "personas",
    "timestamp": "2025-10-16T10:05:00Z"
  }
]
```

- GET /lecturas: listado de lecturas con filtros opcionales:
 - sensorId, ubicacionId.
 - from, to (ISO-8601).
 - limit (máximo 1000 por defecto).

Validación y errores

- Cada lectura valida contra lectura.schema.json.
- Códigos de estado:
 - 200 éxito.
 - 400 parámetros inválidos (fechas o límites).
 - 500 datos no válidos según schema.

Datos semilla

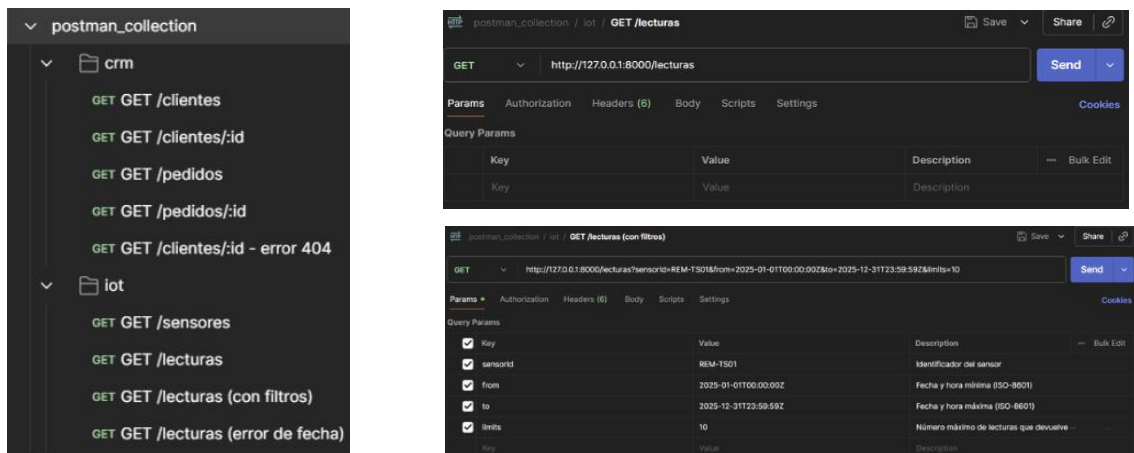
- sensores.json: 3 sensores distintos:
 - REM-TS01 (aforo)
 - REM-TC02 (vibración)
 - REM-TS01-TEMP (temperatura)
- lecturas.json: ≥5 lecturas por sensor, con valor, unidad y timestamp ISO-8601.

Consideraciones

- Filtros aplicados en orden: sensorId → ubicacionId → from → to.
- Determinismo asegurado mediante datos estáticos.
- Lógica de filtro de fechas aplicada
- Todas las fechas convertidas a UTC (Z).

4. Integración y pruebas

- Se probaron todos los endpoints con Postman, exportando la colección a /docs/postman_collection.json.
- Se verificó que todas las respuestas cumplen sus respectivos JSON Schema.
- La paginación y los filtros funcionan correctamente para CRM e IoT.
- Todos los datos son deterministas y reproducibles.



5. Errores y soluciones

1. Detectamos lecturas incompletas en el IoT: añadimos las lecturas que faltaban en el JSON para que coincidieran con todos los sensores.
2. Vimos que algunos sensorId no coincidían: revisamos y corregimos los IDs para asegurar que cada lectura perteneciera a un sensor válido.
3. Encontramos sensores sin ubicacionId: completamos el campo para permitir el filtrado por ubicación sin errores.
4. La paginación del endpoint /clientes no funcionaba bien: ajustamos los índices de paginación para que devolviera exactamente el número de clientes solicitado.
5. Había pedidos con clienteld que no existían: revisamos los datos y actualizamos los pedidos para que todos apuntaran a clientes reales del CRM.
6. El servicio IoT arrancaba en puertos diferentes según la documentación: unificamos el puerto (8002) y corregimos tanto el código como la guía de ejecución.

6. Conclusión

Hemos desarrollado una primera versión completa de los dos microservicios del proyecto, cumpliendo todo lo que se pedía en el Tema 2. El microservicio CRM gestiona clientes y pedidos con búsqueda, filtros y paginación. El servicio IoT ofrece sensores y lecturas con filtros y validación de fechas en formato ISO-8601. Además, ambos microservicios validan sus respuestas usando JSON Schema para asegurarnos de que los datos estén siempre bien formados. Con esta base ya implementada, quedamos preparados para el siguiente Tema, donde integraremos ambas APIs en un único endpoint unificado.